DSA4213 Assignment 1: Language Models, Embeddings, and Representations

Shavonne Lim Jing Ning | A027761W | GitHub link:

1. Introduction and setup

The main question of this project is whether **better next-word prediction (lower perplexity) leads to better representations for a downstream task**. To answer this, I compare four language-model families on one corpus (Part I) and three input-embedding settings on the LSTM (Part II). I then use all the trained models as frozen feature extractors for AG News topic classification (Part III).

1.1 Language-modelling data

WikiText-2 (raw) was used with its standard splits (Merity et al., 2016):

- Lowercase the text (to match GloVe), split it on whitespace (the raw text already separates punctuation and writes hyphens as @-@) and add `<eos>` at the end of each line. This gives 2.08M / 216k / 244k tokens for train/validation/test.
- Vocabulary was built from the training split only, keeping words that appear at least 3 times. This gives V = 29,572, and every other word becomes `<unk>`. The OOV (Out-Of-Vocabulary) rates are 2.25% / 4.72% / 5.62%. Val/test are higher because they contain names and terms that never appear in train, which is expected when vocabulary is built without looking at them.
- Section-heading markup (`= = Gameplay = =`) is kept, so `= = =` is the most frequent trigram. Every model sees the same text, so this does not affect the comparison.

1.2 Downstream data

AG News (Zhang et al., 2015) has 4 balanced topics (World, Sports, Business, Sci/Tech). I sample 10,000 training and 2,000 validation examples from the AG News training set (seed 42, no overlap) and use the full 7,600-example test set. I clean the HTML codes, tokenise the text with a regex that matches the WikiText format and encode it with the WikiText vocabulary (OOV 7.19%). I then truncate each text to 64 tokens, which cuts 6.8% of the texts.

1.3 Shared neural setup

All neural models use 100-d input embeddings (chosen to match GloVe 100d), width 256, 2 layers, dropout 0.3 and untied input/output embeddings. Training uses Adam (learning rate 1e-3), batch size 32, gradient clipping at 1.0 and 10 epochs, and I keep the checkpoint with the lowest validation loss. The text is cut into 64-token windows from 32 contiguous streams, and each window is processed independently. So every neural model sees at most 64 previous tokens, while the n-gram sees 2. There is no padding, so no padding tokens enter the loss. I set seed 42 before building each model.

1.4 Evaluation

Test cross-entropy per token (CE) and perplexity (PPL = exp(CE)) are reported, computed on the same test split for every model. All timings are from one Tesla T4 GPU on Google Colab, with `torch.cuda.synchronize()` called before the clock is read.

2. Part I: Language model comparison

2.1 Models

The n-gram is an interpolated trigram, $P = \lambda1 \cdot P_uni + \lambda2 \cdot P_bi + \lambda3 \cdot P_tri$. The unigram uses add-one smoothing so it is never zero, and I chose $\lambda = (0.30, 0.55, 0.15)$ by grid search on the validation set. The RNN (tanh) and LSTM share one model class that differs only in the recurrent cell. The Transformer projects the 100-d embeddings to 256 and adds learned positional embeddings, then uses 4 heads, a feed-forward size of 512, pre-LayerNorm and a causal mask. Causality was checked directly: changing the token at position 40 leaves the output at positions 0-39 unchanged. All three neural models have about 11M parameters, and 92-98% of these sit in the embedding and output layers, so model size mostly reflects the vocabulary size.

Model	Test CE	Test PPL	Parameters	Train time (10 ep.)	Inference throughput (tok/s)
Interpolated trigram	5.626	277.5	2.0M n-grams	3.6 s (CPU)	497k (CPU)

Model	Test CE	Test PPL	Parameters	Train time (10 ep.)	Inference throughput (tok/s)
RNN	5.301	200.5	10.78M	411 s	146k
LSTM	5.183	178.2	11.45M	478 s	114k
Transformer	5.180	177.7	11.65M	435 s	142k

Table 1. Test results on WikiText-2. Inference workload: score the full test set (batch 32, 64-token windows), with one warm-up run followed by the mean of 3 timed runs.

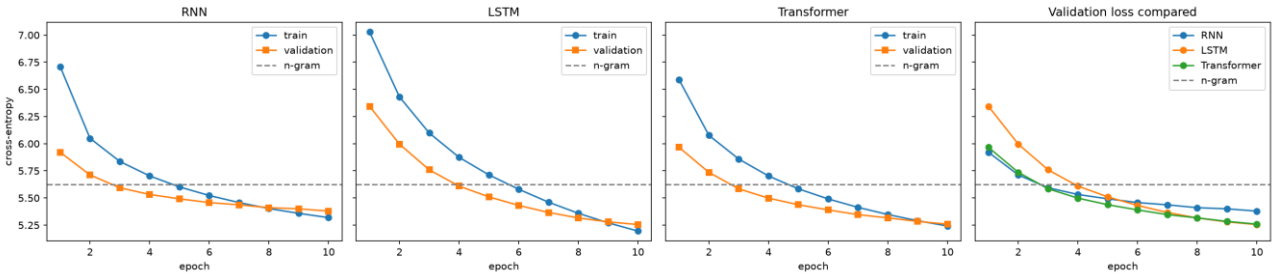


Figure 1. Training and validation cross-entropy per epoch. The dashed line is the n-gram's test cross-entropy.

2.2 Observations

- **Quality:** All neural models beat n-gram, by 28% (RNN) to 36% (LSTM) in perplexity. The LSTM and Transformer are essentially tied (178.2 vs 177.7). With a single seed, I treat a 0.5 PPL gap as no difference. Both clearly beat RNN.
- **Learning behaviour:** The Transformer has the lowest validation loss early on (epoch 1: 5.97 vs 6.34 for the LSTM) and is below the n-gram line by epoch 3. The LSTM starts slowest and catches up at around epoch 8. The RNN levels off after epoch 6 (validation loss 5.46 to 5.38 over epochs 6-10), and its train loss falls below its validation loss from epoch 8, an early sign of overfitting. The LSTM and Transformer are still improving at epoch 10, which is their best epoch. Train loss sits above validation loss early on because dropout is only active during training and the train loss is averaged over the whole epoch.
- **Cost:** The n-gram trains about 100 times faster (3.6s) but has 36% higher perplexity than the LSTM. The Transformer is faster than the LSTM in training (about 44 vs 48 s per epoch) and in inference (142k vs 114k toks/s), even though it has slightly more parameters. The n-gram is timed on the CPU and the neural models on the GPU, so their speeds are not directly comparable.
- **Generated text (Table 2):** With greedy decoding every model repeats itself. The n-gram falls into an exact loop ("the the grand slam tournament , and the the grand slam tournament ..."), and the neural models loop on frequent phrases ("the city of the city of", "the first time of the") and on heading markup ("= = = ="). With sampling, the n-gram produces broken phrases with doubled function words (" a of bella center and a the the"). The neural models are locally fluent ("the fort remained in the early 20th century"), but all of them drift off topic within 30 tokens; for example, the Transformer's "in 1990 , the" continuation turns into a hurricane description (Appendix B).

Model	Prompt "the city of": sampling (T = 0.8, top-k = 40)	Greedy
n-gram	the season was . <eos> and the = = that and was the the more " . the " , she was a of bella center and a the the	the the grand slam tournament , and the the grand slam tournament , and the the grand slam tournament , ...
RNN	the pacific ocean (the southern terminus from september 17 , 2005) . a storm of the area is more than 1 @.@ 4 m ⊙ , and the	the city 's first division . <eos> = = = = = <eos> the first time of the city 's first season , ...
LSTM	the northern union . the area was demolished . when the company was a new american force , the fort remained in the early 20th century , which was the	the city . the first time of the church was the first time of the war , and the first time of ...
Transformer	the central terminus . the missouri river were known locally known as the missouri and was the first creek was recorded to the eastern terminus of central pacific ocean .	the city of the city of the city of the city of the city of the city . the city was ...

Table 2. Continuations of the same prompt (30 new tokens, seed 42; <unk> banned for all models; neural context cropped to 64 tokens). More prompts are in Appendix B.

2.3 Interpretation

The LSTM's advantage over the RNN is consistent with its gates letting it use more of the 64-token window while the RNN's state fades (vanishing gradients). This is a hypothesis as I did not measure how much context each model actually uses. One possible reason the LSTM and Transformer tie is that a 64-token context and about 11M parameters give the Transformer's direct access to every earlier token little room to help. Its speed advantage fits with attention processing all 64 positions in parallel instead of step by step. The n-gram's greedy loop is not a hypothesis as a greedy trigram's only state is its last two tokens, so once a pair repeats, the output must cycle. Five prompts are anecdotal, so I use the samples only to illustrate the differences, not to rank the models. Main tradeoff is that the neural models buy a 28-36% lower perplexity at roughly 100 times the training cost. Among them, the Transformer gives LSTM-level quality at lower cost. All rankings hold for a 10-epoch budget, not at convergence.

3. Part II: Embedding variants and ablation

I use the LSTM and change only its input embedding table, keeping the same data, vocabulary, architecture, hyperparameters and 10-epoch budget. Calling `set_seed()` before each model also gives all three the same starting weights in the LSTM and output layers.

- **A: random, trainable** – The Part I LSTM, reused unchanged.
- **B: my own word2vec, frozen** – Skip-gram, 100-d, window 5, 5 negative samples, 5 epochs, trained on the WikiText-2 training split only, after mapping rare words to <unk>. It covers 100% of vocabulary, and training took 61.9s on CPU.
- **C**: **GloVe 6B 100d, frozen** (Pennington et al., 2014; Wikipedia + Gigaword newswire, about 6B tokens) – It covers 96.2% of the vocabulary types and 95.1% of the training tokens. The most frequent missing items are <unk>, <eos>, @-@, @.@ and @,@, so about 5% of input positions get a fixed random vector. The rest are tokenization differences ('t) and rare names. Missing words are initialized from N(μ, σ) of the found vectors.

In B and C the whole embedding table has `requires_grad=False`, which leaves 8.49M trainable parameters instead of 11.45M. After training I checked that the table was unchanged.

Setting	Test PPL	Val PPL ep 1 / 5 / 10	Epochs val got worse	Final train–val gap	LM train time	Embedding cost
A: random, trainable	178.2	569 / 247 / 192	0	−0.060	478 s	learned jointly
B: word2vec, frozen	**165.2**	**598 / 216 / 177**	**0**	**−0.044**	**443 s**	**61.9 s (CPU)**
C: GloVe, frozen	172.9	723 / 245 / 186	0	+0.039	445 s	none (pre-trained)

Table 3. Embedding ablation on the LSTM. The cost of training the embeddings is reported separately from LM training.

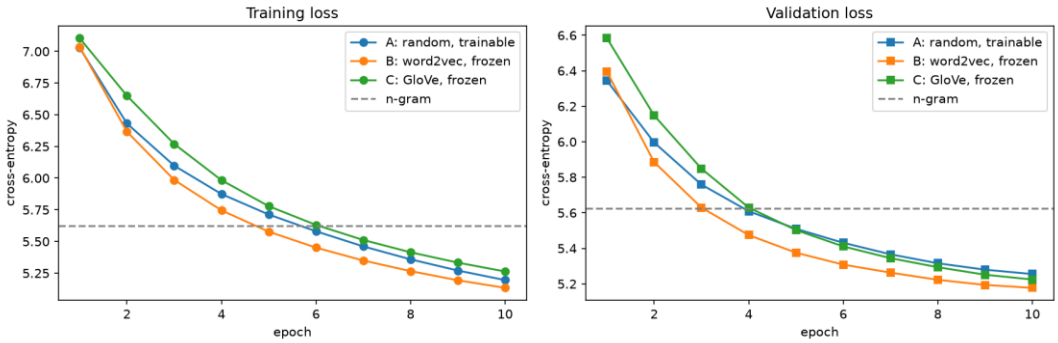


Figure 2. Training and validation loss for the three embedding settings.

Word	Nearest neighbours: my word2vec (WikiText-2, 2M tokens)	Nearest neighbours: GloVe (6B tokens)
game	games, franchise, crush, console, multiplayer, player	games, play, season, player, players, match
album	songs, studio, yankovic, carey, remix, soundtrack	song, albums, compilation, soundtrack, ep, songs

Table 4. Nearest neighbours by cosine similarity.

3.1 Observations

- Both frozen settings beat the trainable random embeddings, and my in-domain word2vec beats GloVe (165.2 vs 172.9), even though GloVe was trained on about 3,000 times more text.
- Convergence differs. B is ahead from epoch 2 onwards. C starts worst (validation PPL 723 at epoch 1) but passes A at around epoch 5. A starts best and then improves more slowly.
- Stability is the same for all three: the validation loss improved every epoch and the curves are smooth. Only C has a slightly positive train-val gap (+0.039).
- The frozen models train about 7% faster because no embedding gradients are computed. Counting the word2vec cost, B's total (505s) is slightly above A's (478s).
- The generated text does not clearly separate the settings. Greedy decoding collapses into = = = for all three, and the sampled text shows similar fluency, with occasional tokens repeats in C ("hot hot hot"). PPL is the clearer signal here.

3.2 Interpretation

- **Trainability vs head start:** With 10 epochs, Setting A has to learn the embeddings and the LSTM at the same time, and rare words get few updates. B and C start from vectors that already encode word similarity. Freezing also removes 2.96M trainable parameters, which may act as regularization. All three were still improving at epoch 10, so A might catch up with longer training. My conclusion holds for this budget only.
- **Extra data vs domain match:** GloVe's 6B extra tokens did not help perplexity here. Table 4 shows why this is plausible. My word2vec learned WikiText's meaning of game (video games: console, multiplayer) and its artists (yankovic, carey), while GloVe encodes the news/sports meaning (season, match). Frozen GloVe cannot adapt to the corpus. Its missing special tokens are also some of the most frequent inputs. GloVe's slow start may come from its different training objective and scale, which the LSTM has to adapt to first. Main takeaway is that when embeddings are frozen, matching the corpus's tokenization and word usage mattered more than the amount of data the embeddings were trained on.

4. Part III: Downstream representation analysis

4.1 Representations

I run each text through a frozen LM and take the top-layer hidden states (256-d) at every position. I then pool them in one of two ways: the mean over the real tokens, with padding excluded, or the hidden state at the last real token. Texts are padded on the right. This is safe because every LM is causal, so padding after a token cannot change that token's state, and a mask leaves the padding out of the mean. As a baseline that uses no LM, I also average the 100-d input embedding vectors of each text ("bag-of-embeddings") for the A, B and C tables.

4.2 Classifier

The same pipeline for every representation: standardize the features, then logistic regression. I choose C from {0.01, 0.1, 1, 10} by validation accuracy and use the test set once. The LMs are never fine-tuned.

Representation	LM test PPL	Val acc	Test acc	Test macro-F1
Bag-of-embeddings, GloVe (no LM)	–	89.6%	87.2%	0.871
LSTM-C (GloVe, frozen), mean	172.9	84.1%	82.4%	0.823
Bag-of-embeddings, word2vec (no LM)	–	78.9%	77.6%	0.775
LSTM-B (word2vec, frozen), mean	165.2	76.4%	76.7%	0.766
Transformer, mean	177.7	73.0%	72.8%	0.726
RNN, mean	200.5	65.2%	66.1%	0.658

Representation	LM test PPL	Val acc	Test acc	Test macro-F1
LSTM-A (random), mean	178.2	64.3%	64.4%	0.642
Bag-of-embeddings, learned A (no LM)	–	61.5%	62.0%	0.619
LSTM-A (random), last state	178.2	56.8%	58.5%	0.581

Table 5. AG News results (7,600 test texts, balanced; chance = 25%), sorted by test accuracy.

4.3 Observations

- **The embedding choice matters most:** Among the LM representations, the two frozen-embedding LSTMs rank first and second. LSTM-C beats LSTM-A by 18 points, even though they share an architecture and have similar perplexity.
- **The LM layers add little on top of the input embeddings:** Averaging the GloVe vectors directly (87.2%) beats every LM representation, and passing them through the trained LSTM lowers accuracy to 82.4% (-4.8 points). For word2vec the change is -0.9 points. Only with the weakest (learned) embeddings does the LSTM help (+2.4 points). The embedding ranking (GloVe > word2vec > learned) is the same with or without the LM.
- **Perplexity only loosely predicts the downstream ranking:** Spearman's $\rho = -0.80$ ($p = 0.10$, $n = 5$ mean-pooled models), so the direction agrees but the evidence is weak. There are clear reversals. LSTM-A and the Transformer have almost the same perplexity (178.2 vs 177.7) but are 8.4 accuracy points apart. Word2vec has the better perplexity, but GloVe has the better accuracy. The RNN has the worst perplexity but slightly beats LSTM-A.
- **Mean pooling beats last-state pooling** for LSTM-A (64.4% vs 58.5%).

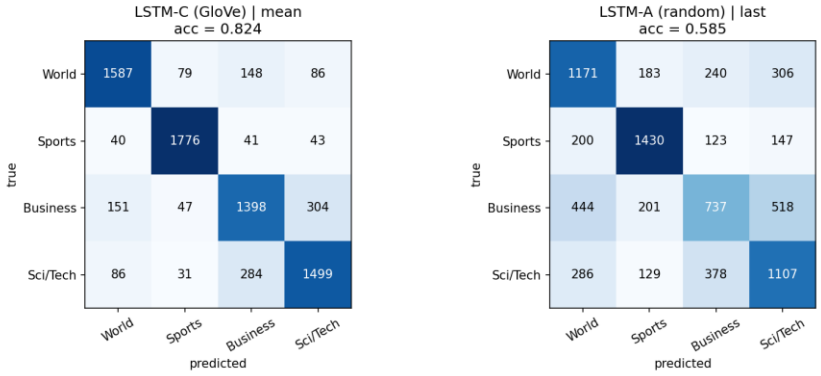


Figure 3. Confusion matrices for the best LM representation (left) and the worst representation (right).

Errors (LSTM-C, 1,340 of 7,600 wrong). Business ↔ Sci/Tech accounts for 588 errors (44%) and Business ↔ World for 299 (22%), while Sports is the easiest class (1,776 of 1,900 correct). Missing words do not explain the errors: the OOV rate is 7.09% in misclassified texts and 7.17% in correct ones. Some errors look like genuinely ambiguous or arguably mislabelled texts. For example, *"Pakistan down India … Olympic men's field hockey"* is labelled World but predicted Sports, and a Nepal tourism blockade is labelled Business but predicted World. Others come from words of another topic, such as a track-and-field interview full of media terms that is predicted as Sci/Tech. A few have no clear cause, such as a US–Australia free-trade deadline predicted as Sports.

4.4 Interpretation

Part III reverses Part II's result. GloVe lost to word2vec on WikiText perplexity, yet it gives the best features for news classification. A plausible reason is that GloVe's Gigaword newswire data matches AG News, while my word2vec and LMs only saw Wikipedia. So the value of extra pre-training data depends on whether its domain matches the task. My hypothesis for why the LM layers do not help is that next-word prediction on WikiText mostly rewards local, syntactic information, so the hidden states may transform topic information that a linear classifier could read straight from the input vectors. Last-state pooling mostly reflects the final few tokens, which are often generic ("… said on tuesday ."), while mean pooling also covers the headline.

5. Conclusion, limitations and lessons

The neural LMs clearly beat the n-gram (28-36% lower perplexity at about 100 times the cost), and the LM and downstream rankings broadly agree. But perplexity is a poor guide to representation quality. Two models with near-identical perplexity differed by 8.4 accuracy points, and averaging frozen GloVe vectors without any LM beat every LM representation. The embedding initialization and the domain it was trained on mattered more than perplexity. The same GloVe vectors were worse for WikiText perplexity (Part II) and best for news classification (Part III).

5.1 Limitations

- I used a single seed, so small gaps (such as LSTM vs Transformer) are within run-to-run noise.
- All comparisons use an equal 10-epoch budget, and most models had not converged so the rankings may change with longer training.
- The corpus is small (2M tokens) and the context is limited to 64 tokens, which may hide the Transformer's long-range advantage.
- The n-gram is timed on the CPU and the neural models on the GPU.
- The perplexity-accuracy correlation uses only n = 5 models. Part III uses a linear probe on frozen features only.
- Some AG News texts still contain HTML tags (`<b>`) and `quot;` codes after cleaning, and word2vec vectors differ very slightly across machines (maximum difference 0.002 between Colab and my laptop), so I save and reuse the trained model.

5.2 Lessons

First, an equal-budget comparison answers a different question than a comparison at convergence, and it needs to be reported as such. Second, a simple baseline can change a conclusion: the bag-of-embeddings baseline showed that most of the downstream signal came from the embeddings, not the LM. Third, checking error cases tested a hypothesis directly. While I expected missing words to cause errors, the OOV comparison showed they did not.

References

1. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
2. Lhoest, Q., et al. (2021). Datasets: A community library for natural language processing. *EMNLP 2021: System Demonstrations*.
3. Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2016). Pointer sentinel mixture models. arXiv:1609.07843. (WikiText-2)
4. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
5. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS 2019*.
6. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.
7. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *EMNLP 2014*.
8. Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *LREC 2010 Workshop on New Challenges for NLP Frameworks*. (gensim)
9. Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 2017*.
10. Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *NeurIPS 2015*. (AG News)

Appendix A: Hyperparameters and reproducibility

Setting	Value
Vocabulary	train split only, frequency ≥ 3, lowercased; V = 29,572 (<unk>, <eos> included)
n-gram	interpolated trigram, add-one unigram; λ = (0.30, 0.55, 0.15) from a 0.05-step grid on validation
Embedding / width / layers	100 / 256 / 2; untied input and output embeddings
Dropout	0.3 on embeddings, between layers and before the output layer
RNN / LSTM	`nn.RNN` (tanh) / `nn.LSTM`, zero initial state per 64-token window
Transformer	linear 100→256, learned positions (64), 4 heads, feed-forward 512, pre-LayerNorm + final LayerNorm, causal mask
Training	Adam, lr 1e-3, batch 32, clip 1.0, 10 epochs, window order shuffled each epoch, best-validation checkpoint
word2vec	skip-gram, 100-d, window 5, negative 5, min_count 1, 5 epochs, workers 1, seed 42
Generation	5 prompts × 30 tokens; sampling T = 0.8, top-k 40, seed 42 + prompt index; greedy; <unk> banned
Part III	64-token truncation; StandardScaler + LogisticRegression (max_iter 3000), C ∈ {0.01, 0.1, 1, 10} chosen on validation
Seeds / hardware	seed 42 for Python, NumPy, PyTorch before every model; Google Colab Tesla T4 (Parts I–II), Colab CPU (Part III extraction)

Causality check (Transformer): In an untrained Transformer, changing only the token at position 40 left the outputs at positions 0–39 identical (`torch.allclose`, tolerance 1e-5) and changed the outputs from position 40 onwards. The notebooks, README (with data access, commands, configurations and seeds) and all saved results are in the code submission.

Appendix B: Additional generated text

Model	Prompt "in 1990 , the": sampling	Prompt "in 1990 , the": greedy
n-gram	second album , " it was a , which a local of the album was created to the first part of the the today) the the , in a	the grand slam tournament , and the the grand slam tournament , and the the grand slam tournament , …
RNN	area was a first largest @-@ class area at the university of the west pacific ocean . the road continued northward into the eastern pacific area . however , it	first time of the first time , the first time of the game 's first season , the hokies was released in the united states , …
LSTM	last series of the american theatre of the game of the american song , which aired by the new york 's film . <eos> = = = film = =	first @-@ selling @-@ time @-@ old @-@ old @-@ old @-@ old @-@ old …
Transformer	storm was the third part of the first @-@ day trough on the first . it was the hurricane struck a tropical cyclone and intensified , and in the storm	first time of the first time , the first time of the season . <eos> = = = = = = = = …

Table B1. Continuations of the prompt "in 1990 , the". All 5 prompts for all models, including the Part II settings, are in `results/generations.txt` and `results/generations_part2.txt`.

Appendix C: Use of AI tools

I used Claude as a step-by-step guide while doing this assignment. It explained the assignment to me in detail, guided me through the coding and helped me debug errors. I ran all experiments myself in Google Colab, checked the code and outputs, and interpreted the results. I also tested that the notebooks run locally in VS Code, reusing the saved Colab results.